

Práctica 2: Integración DevSecOps CI/CD

Nombre: Elmer Alan Cornejo Quito

Respositorio: [<https://github.com/AlanCornejoQ/Practica2>]

1. Justificación Técnica de Decisiones (Pipeline DevSecOps)

En el presente repositorio se ha diseñado e implementado un pipeline CI/CD automatizado mediante gitHub Actions. El objetivo es garantizar que el código solo avance hacia la fase de despliegue si cumple con estrictos criterios funcionales y de seguridad, aplicando un enfoque DevSecOps.

A continuación, la justificación técnica de cada etapa implementada:

- **A. Instalación Reproducible (npm ci)**
 - **Fase:** Build (Construcción).
 - **Riesgo mitigado:** Inconsistencias entre entornos de desarrollo y producción.
 - **Necesidad:** Aunque la app funcione localmente, **npm ci** es vital para bloquear versiones de dependencias no probadas que podrían romper despliegues futuros.
- **B. Análisis de Calidad (ESLint)**
 - **Fase:** Code (Calidad continua).
 - **Riesgo mitigado:** Deuda técnica, variables sin uso y malas prácticas.
 - **Necesidad:** Un código funcional no siempre es mantenible; el linter estandariza el código para evitar errores lógicos a largo plazo.
- **C. Pruebas Unitarias (Jest / npm test)**
 - **Fase:** Test.
 - **Riesgo mitigado:** Regresiones de software.
 - **Necesidad:** Actúa como red de seguridad para garantizar que los nuevos parches no rompan la funcionalidad actual.
- **D. Seguridad de Dependencias - SCA (npm audit)**
 - **Fase:** Secure (SCA).
 - **Riesgo mitigado:** Inclusión de librerías de terceros con vulnerabilidades críticas (CVEs).
 - **Necesidad:** Nuestro código base puede ser seguro, pero una sola dependencia externa comprometida expone todo el sistema.
- **E. Seguridad Estática - SAST (Semgrep)**
 - **Fase:** Secure (SAST).
 - **Riesgo mitigado:** Vulnerabilidades en código propio (inyecciones, secretos hardcodeados).
 - **Necesidad:** Analiza la lógica de negocio en busca de patrones inseguros antes de generar el contenedor.
- **F. Seguridad de Contenedores (Trivy)**

- **Fase:** Package / Release.
- **Riesgo mitigado:** Fallos de seguridad en el SO base de la imagen Docker.
- **Necesidad:** Garantiza que la infraestructura inmutable (como Alpine o Node base) no arrastre vulnerabilidades al entorno de producción.

- **G. Smoke Test y DAST (curl)**

- **Fase:** Deploy / Test Dinámico.
- **Riesgo mitigado:** Endpoints desprotegidos y fallos de conexión de red.
- **Necesidad:** Comprueba en tiempo de ejecución que los microservicios se comunican y que el API Gateway bloquea efectivamente accesos no autorizados sin un JWT válido.

2. Evidencias de Ejecución

A continuación, se adjuntan las evidencias de que el pipeline funciona correctamente, bloqueando el código cuando detecta riesgos y permitiendo el paso cuando se cumplen los estándares:

Historial de Ejecuciones

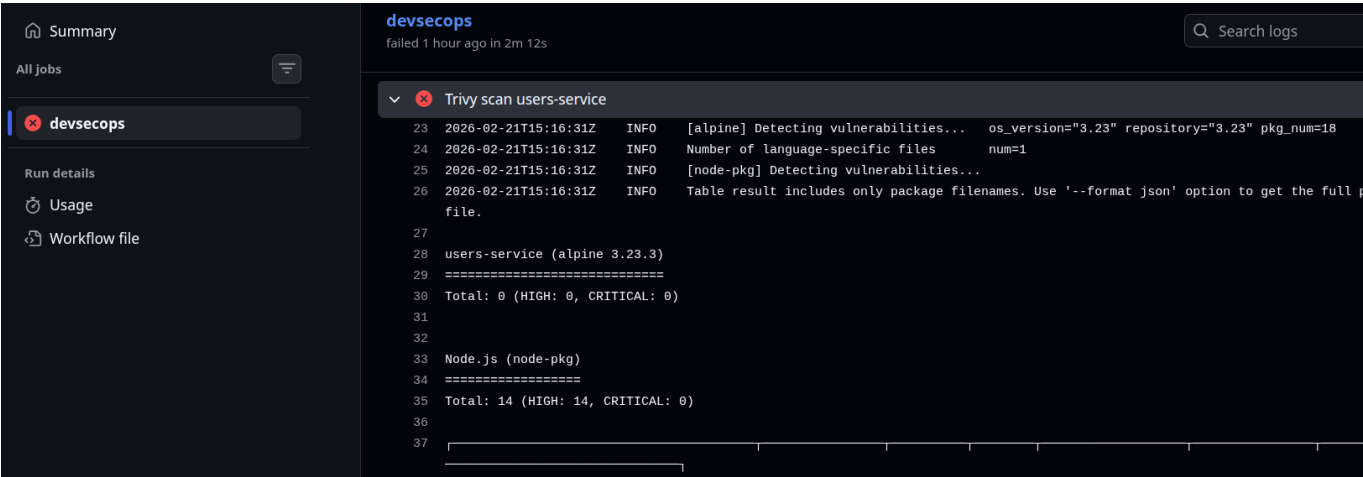
En esta captura se observa el proceso iterativo, evidenciando cómo los *gates* detuvieron el flujo inicialmente y cómo, tras las correcciones, el pipeline finalizó con éxito.

The screenshot shows the GitHub Actions interface. On the left, the 'Actions' tab is selected, displaying the 'DevSecOps CI/CD Pipeline' workflow. The main area shows a list of 6 workflow runs. The first run, 'fix: completo pipeline con linting, SCA y purebas DAST', is successful (green checkmark). The subsequent five runs are failed (red X), each with a specific error message related to security and linting fixes. The runs are ordered chronologically from bottom to top.

Run Status	Run Title	Commit	Pushed by	Branch
Success	fix: completo pipeline con linting, SCA y purebas DAST	794445d	AlanCornejoQ	main
Failure	fix: corr errores eslint en tests y configuracion	11b6562	AlanCornejoQ	main
Failure	fix: actualizo directivas de eslint soporte flat config en test	e961077	AlanCornejoQ	main
Failure	fix: ajuste de trivy a crititcal para no romper dependencias	9ecf223	AlanCornejoQ	main
Failure	fix: complementar strict fail en SCA y versionado en docker	3ed4ec4	AlanCornejoQ	main
Failure	fix: corrijo rutas relativas XD	4e6932e	AlanCornejoQ	main

Evidencia de Gate Fallido (Bloqueo de seguridad)

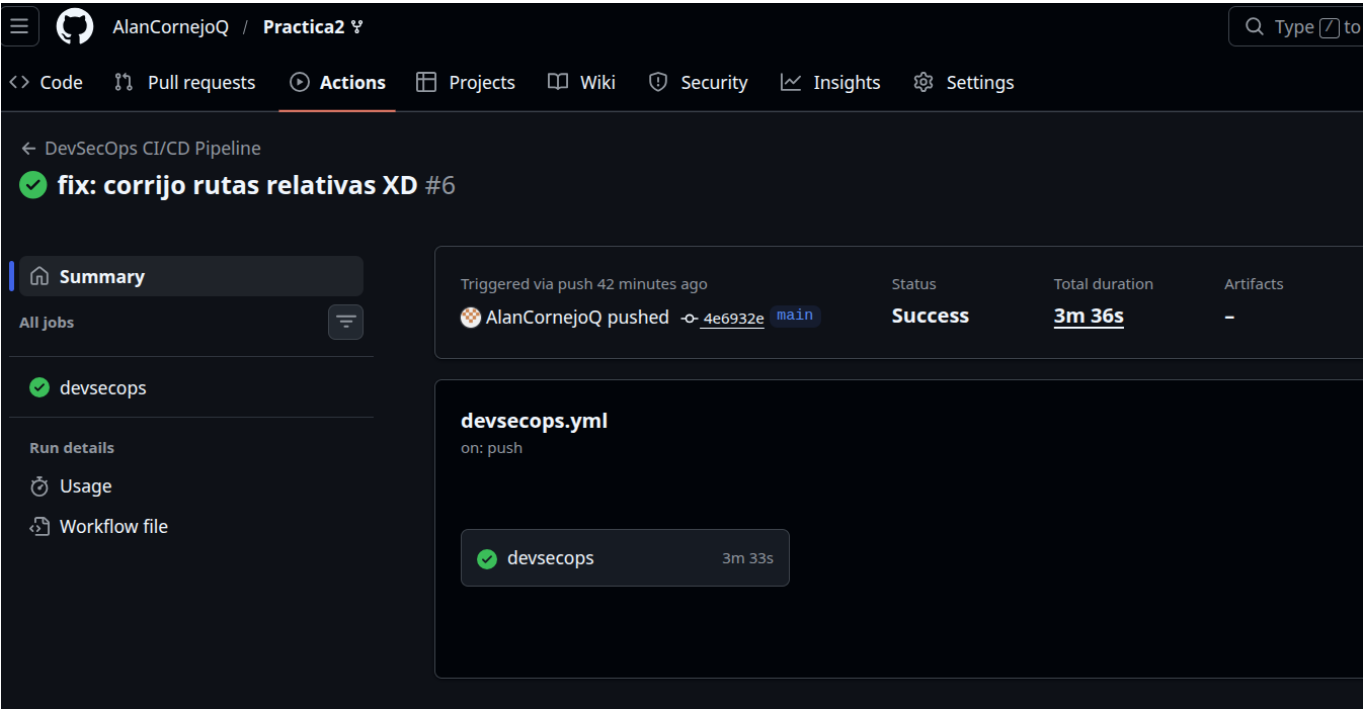
El pipeline bloqueó exitosamente el despliegue debido a configuraciones estrictas, demostrando la mitigación automatizada de deuda técnica/vulnerabilidades.



(Enlace a la ejecución fallida: [https://github.com/AlanCornejoQ/Practica2/actions/runs/22259153668])

Ejecución Exitosa (Pipeline DevSecOps aprobado)

Flujo completo tras resolver los hallazgos de seguridad y formato. Todas las etapas de DevSecOps finalizaron correctamente.



(Enlace a la ejecución exitosa: [https://github.com/AlanCornejoQ/Practica2/actions/runs/22259689372])